Tablo Düzeltici

Sitenin içi nasıl çalışır?
Dosya okumadan indirmeye kadar tüm kodun ayrıntılı anlatımı

Uygulama	React 18 + Vite 5, tamamen tarayıcıda çalışan tek sayfalık site
Kütüphane	SheetJS (xlsx 0.20.3): Excel/CSV okuma ve yazma
Kod	22 dosya, yaklaşık 3.700 satır (motor + arayüz + testler)
Sunucu	Yok. Dosya hiçbir yere gönderilmez, hiçbir yere kaydedilmez
Kaynak kod	github.com/yusufmertd (kayit-eslestirme deposu)

İçindekiler

1. Büyük resim: site ne yapıyor? 4

1.1 Veri akışı tek bakışta . . . 4

1.2 Klasör yapısı . . . 4

1.3 Neden React ve Vite? . . . 5

2. Dosya nasıl okunuyor? (io.js) 6

2.1 Baytları belleğe almak . . . 6

2.2 Excel mi, CSV mi? . . . 6

2.3 Sayfayı tabloya çevirmek . . . 6

3. Metin temelleri (text.js) 7

3.1 Türkçe büyük/küçük harf . . . 7

3.2 fold: karşılaştırma anahtarı . . . 7

3.3 Düzenleme mesafesi (Damerau-Levenshtein) . . . 7

3.4 Kaç hataya izin verilir? . . . 7

4. Sözlükler ve dosyadan öğrenme 8

4.1 Hazır sözlükler (dictionaries.js) . . . 8

4.2 Arama tablosu (buildLookup) . . . 8

4.3 Bir değeri sözlükte aramak (lookup) . . . 8

4.4 Dosyanın kendisinden öğrenme (buildContext) . . . 8

5. Sütun türünü tanıma (categories.js) 10

5.1 Kategori tanımı . . . 10

5.2 Tanıma sırası (detectColumnType) . . . 10

6. Hücre temizleyicileri 11

6.1 E-posta alan adı düzeltme (fixDomain) . . . 11

6.2 Gerçek kişi e-postası (humanizeEmail) . . . 11

7. Sütun içi tek yazım (harmonizeColumn) 13

8. Tekrar eden kayıtları bulma (matcher.js) 13

8.1 Kayıt özeti . . . 13

8.2 Ad benzerliği (nameSimilarity) . . . 13

8.3 İki kaydı karşılaştırma kuralları (compareRecords) . . . 13

8.4 Gruplama (union-find) . . . 13

9. Ana işlem hattı (pipeline.js · processTable) 15

9.1 Adım adım . . . 15

9.2 Üç mod . . . 16

9.3 Kararlar (decisions) nesnesi . . . 16

10. Arayüz (React bileşenleri) 17

10.1 App.jsx: durum ve akış . . . 17

10.2 Assistant.jsx: soru balonu . . . 17
10.3 ResultTable.jsx: temiz tablo . . . 17
10.4 Groups.jsx, Settings.jsx, Help.jsx . . . 18
10.5 Açıklama balonları ve tema (styles.css) . . . 18
11. İndirme (io.js) 19
11.1 İndirilecek tabloyu hazırlamak (tableFor) . . . 19
11.2 Excel ve CSV . . . 19
12. Gizlilik: dosya neden hiçbir yerde kalmıyor? 19
13. Testler 20
13.1 Motor testi (npm test → tests/evaluate.js) . . . 20
13.2 Sentetik veri üreticisi (tests/generate.js) . . . 20
13.3 Tarayıcı testi (tests/tarayici-testi.mjs) . . . 20
13.4 Testlerin yakaladığı hatalar (örnekler) . . . 20
14. Yayınlama 21
15. Baştan sona bir örnek 22
16. Ek: dosya dosya önemli fonksiyonlar 22

1. Büyük resim: site ne yapıyor?

Tablo Düzeltici, içine atılan bir Excel ya da CSV tablosunu **okur**, her sütunun ne tür bilgi taşıdığını **anlar**, her hücreyi o türün kurallarına göre **düzeltir**, aynı kişiye/kayda ait tekrar eden satırları **bulur**, bu satırlar arasında en doğru yazımı **seçer**, emin olamadığı yerde kullanıcıya **sorar** ve sonunda temiz tabloyu tekrar Excel/CSV olarak **indirtir**.

Bütün bu işlemler kullanıcının kendi tarayıcısında, JavaScript ile yapılır. Vercel sadece sitenin dosyalarını (HTML, JS, CSS) dağıtır; sitede hazır örnek dosya yoktur; yüklenen tablo Vercel'e ya da başka bir sunucuya hiç gitmez.

1.1 Veri akışı tek bakışta

```
Kullanıcı dosyayı sürükler
         │
         ▼
io.js  readFile()          ArrayBuffer → SheetJS → sayfalar [{name, headers, rows}]
         │
         ▼
App.jsx  sheets (React state, sadece bellekte)
         │   her değişiklikte (karar, mod, ayar) yeniden hesaplanır
         ▼
pipeline.js  processTable(headers, rows, decisions)
  1) categories.js   detectColumnType   → her sütunun türü (58 kategori)
  2) fields.js / categories.js CLEANERS → her hücre temizlenir
  3) harmonizeColumn                    → sütun içinde tek yazım
  4) matcher.js  findPairs + buildGroups → tekrar eden kayıtlar
  5) grup içi en iyi değer               → pickBest / longestValue / bestEmail
  6) e-posta onarımı, biçimler           → telefon, tarih, gerçek e-posta
  7) sorular                             → mod'a göre kullanıcıya sorulacaklar
         │
         ▼
Ekran: Assistant (soru balonu) · ResultTable (tablo) · Groups (eşleşen kayıtlar)
         │   kullanıcı cevap verir → decisions güncellenir → 4. adımdan tekrar
         ▼
io.js  downloadXlsx() / downloadCsv()  → Blob → tarayıcının "indir" işlemi
```

1.2 Klasör yapısı

Dosya	Satır	Görevi
src/main.jsx	6	React'i başlatır, App bileşenini sayfaya yerleştirir
src/App.jsx	267	Uygulamanın beyni: durum (state), kararlar, geri al, indirme, sekmeler
src/io.js	84	Dosya okuma (Excel/CSV), indirilecek tabloyu hazırlama, indirme
src/engine/text.js	82	Temel metin araçları: Türkçe küçük harf, fold, düzenleme mesafesi, benzerlik
src/engine/dictionaries.js	630	Sözlükler: 196 üniversite, 119 bölüm, 81 il, 269 ad, 160 soyad, alan adları...
src/engine/fields.js	368	Ad, kurum, bölüm, şehir, e-posta, telefon, adres, tarih temizleyicileri
src/engine/categories.js	330	58 sütun kategorisi, sütun türü tanıma, IBAN/TC/plaka gibi ek temizleyiciler
src/engine/table.js	16	Başlık satırını bulma (üstteki açıklama satırlarını atlama)
src/engine/matcher.js	111	İki kaydı karşılaştırma, çift bulma, gruplama (union-find)
src/engine/pipeline.js	530	Tüm adımları sırayla çalıştıran ana işlem hattı
src/components/*.jsx	498	Arayüz: Upload, Assistant, ResultTable, Groups, Settings, Help
src/styles.css	-	Görünüm: açık/koyu tema, tablo, balonlar, ipuçları, mobil

Dosya	Satır	Görevi
tests/*.js, *.mjs	752	Motor testleri, sentetik veri üretici, yazım testleri, tarayıcı testi

1.3 Neden React ve Vite?

React ekranı "durumdan" çizer: tablo, sorular ve gruplar her zaman o anki verinin bir sonucudur. Kullanıcı bir soruya cevap verince sadece kararlar nesnesi değişir; React her şeyi yeniden hesaplayıp ekranı günceller. Böylece "şu hücreyi elle değiştir, şu listeyi güncelle" gibi dağınık kodlar gerekmez.

Vite geliştirme sunucusu ve derleyicidir. `npm run build` komutu tüm kodu `dist/` klasörüne küçük, statik dosyalar olarak çıkarır; Vercel bu klasörü yayınlar. Sunucu tarafı kod yoktur.

2. Dosya nasıl okunuyor? (io.js)

Kullanıcı dosyayı sürükleyip bıraktığında ya da "Dosya seç" ile seçtiğinde `Upload.jsx` tarayıcının verdiği `File` nesnesini `App.jsx`'teki `handleFile` fonksiyonuna iletir. O da `readFile(file)` çağırır.

2.1 Baytları belleğe almak

```
const buffer = await file.arrayBuffer();
```

Bu satır dosyanın tüm baytlarını tarayıcının belleğine (RAM) alır. Hiçbir ağ isteği yapılmaz. Dosya diske de yazılmaz; sadece JavaScript değişkeni olarak durur ve sayfa kapanınca çöp toplayıcı tarafından silinir.

2.2 Excel mi, CSV mi?

```
const wb = /\.(csv|txt)$/i.test(file.name)
  ? XLSX.read(decodeCsv(buffer), { type: 'string', raw: true })
  : XLSX.read(buffer, { type: 'array' });
```

- **Excel (.xlsx/.xls)** doğrudan SheetJS'e verilir. SheetJS ZIP içindeki XML'i açar, her sayfayı (sheet) çıkarır.
- **CSV** önce metne çevrilir. `decodeCsv` önce katı (fatal) UTF-8 dener; dosya geçersiz UTF-8 ise **windows-1254** (Türkçe Windows kodlaması) ile çözer. Eski Excel'in kaydettiği CSV'lerde "ş, ğ, ı" bu sayede bozulmaz.
- `raw: true`: SheetJS'in CSV'deki değerleri "akıllıca" çevirmesini engeller. Aksi hâlde `0555...` başındaki sıfırı kaybeder, `12/11/1975` Amerikan tarihi (11 Aralık yerine 12 Kasım'ın tersi) sanılırdı. Bu hata testlerde yakalanıp düzeltildi.
- Ayırıcı (virgül, noktalı virgül, sekme) SheetJS tarafından otomatik bulunur.

2.3 Sayfayı tabloya çevirmek

```
const data  = XLSX.utils.sheet_to_json(ws, { header: 1, defval: '', blankrows: false });
const start = findHeaderRow(data);           // üstteki açıklama satırlarını atla
const [headers = [], ...rows] = data.slice(start);
const width = Math.max(headers.length, ...rows.map((r) => r.length));
const pad   = (r) => Array.from({ length: width }, (_, i) => r[i] ?? '');
```

- `header: 1` her satırı basit bir dizi olarak verir: `['Ahmet', 'Yılmaz', '0555...']`.
- `blankrows: false` tamamen boş satırları atar.
- `findHeaderRow` (table.js): ilk 10 satıra bakar. En dolu satırın en az yarısı kadar dolu olan ve tüm hücreleri metin olan ilk satırı başlık kabul eder (genişlik ilk 20 satırdan ölçülür). Böylece "2024 Kayıt Listesi" gibi bir başlık yazısı ya da boş satır üstte olsa bile asıl başlık bulunur.
- `pad`: eksik hücreleri boş metinle doldurur, böylece her satır aynı genişlikte olur.
- Boş başlıklara `Sütun 1, Sütun 2...` adı verilir; tür tanıma o zaman tamamen içerikten yapılır.

Sonuç her sayfa için `{ name, headers, rows, skipped }` nesnesidir. Birden çok sayfalı Excel'de her sayfa ayrı sekme olarak gösterilir ve ayrı işlenir.

3. Metin temelleri (text.js)

Motorun geri kalanı birkaç küçük fonksiyonun üzerine kuruludur. Bunları anlamak bütün sistemi anlamanın anahtarıdır.

3.1 Türkçe büyük/küçük harf

`trLower('İSTANBUL')` → "istanbul", `trUpper('ırmak')` → "IRMAK". JavaScript'in normal `toLowerCase`'u "I" harfini "i" yapar; Türkçede "ı" olmalıdır. Bu yüzden her yerde `toLocaleLowerCase('tr-TR')` kullanılır. `titleCase` her kelimenin ilk harfini büyütür, kalanını küçültür ("AHMET YILMAZ" → "Ahmet Yılmaz").

3.2 fold: karşılaştırma anahtarı

İki yazımın "aynı şey" olup olmadığını anlamak için ikisi de aynı kaba biçime indirilir:

```
fold('Uludağ Üniversitesi')  →  'uludag universitesi'
fold('ULUDAG UNIV.')         →  'uludag univ'
fold('  Ahmet-YILMAZ ')      →  'ahmet yilmaz'
```

Kurallar: Türkçe küçük harf → ç/ğ/ı/ö/ş/ü harfleri c/g/i/o/s/u yapılır → noktalama boşluğa çevrilir → fazla boşluklar silinir. Sözlükteki her kayıt da fold edilmiş hâliyle saklanır, böylece arama hızlı ve harf/aksan farkına dayanıklıdır.

3.3 Düzenleme mesafesi (Damerau-Levenshtein)

`editDistance(a, b)` bir kelimeyi diğerine çevirmek için gereken en az tek harf işlemini sayar: **ekleme, silme, değiştirme ve yan yana iki harfin yer değiştirmesi**. Son işlem sayesinde "Ahmte" → "Ahmet" tek hata sayılır.

a	b	Mesafe	Neden
abmet	ahmet	1	b → h değiştirme
dılmaz	yılmaz	1	d → y değiştirme
ahmte	ahmet	1	t ve e yer değiştirme
istanbl	istanbul	1	u eksik

Hesap klasik dinamik programlama tablosuyla yapılır (satır = a'nın harfleri, sütun = b'nin harfleri). `similarity(a, b) = 1 - mesafe / uzun olanın uzunluğu` ile 0 ile 1 arası bir benzerlik puanı çıkar: "ahmet" ile "abmet" için 1 - 1/5 = 0.8.

3.4 Kaç hataya izin verilir?

```
allowedTypos(word):  uzunluk ≤ 3  → 0 hata   (kısa kelimede tek harf her şeyi değiştirir: "Ali" ≠ "Ala")
                     uzunluk ≤ 7  → 1 hata
                     daha uzun    → 2 hata
```

`closestWord(word, vocab)` sözlükte bu sınırın içinde kalan en yakın kelimeyi arar. **Beraberlik varsa hiçbir şey döndürmez**: "Ayse" hem "Ayşe" hem başka bir isme aynı uzaklıktaysa sistem tahmin yürütmez, kelimeyi olduğu gibi bırakır. Yanlış düzeltme yapmaktansa düzeltmemek tercih edilir; bu bütün tasarımın ana prensibidir.

4. Sözlükler ve dosyadan öğrenme

4.1 Hazır sözlükler (dictionaries.js)

Sözlük	Adet	Örnek
Üniversiteler (ad + kısaltmalar)	196	Orta Doğu Teknik Üniversitesi | ODTÜ | METU
Bölümler	119	Bilgisayar Mühendisliği | Bilg. Müh. | BM
Fakülteler	36	Mühendislik Fakültesi | Müh. Fak.
İller	81	İstanbul, Kahramanmaraş...
Şehir takma adları (ilçe/eski ad → il)	22	Antep → Gaziantep, İzmit → Kocaeli, Antakya → Hatay
Erkek/kadın adları	269	Ahmet, Ayşe, Miraç...
Soyadları	160	Yılmaz, Kaya, Demir...
Unvanlar	29	Prof. Dr., Uzman, Mühendis
Ülkeler / diller / durumlar	45 / 15 / 10	Türkiye, İngilizce, Aktif
E-posta alan adları	19	gmail.com, hotmail.com, outlook.com, yandex.com...
Ortak kelimeler	108	mahallesi, caddesi, sokak, üniversitesi, fakültesi...

Listeler kodda düz metin olarak yazılmıştır, `parseList` bunları satır satır okur: `Ad | Kısaltma1 | Kısaltma2`. Böylece yeni bir üniversite eklemek için sadece bir satır eklemek yeterlidir.

4.2 Arama tablosu (buildLookup)

Her sözlük için bir kez şu yapılar kurulur:

- **names**: fold edilmiş tam ad → doğru yazım (`'uludag universitesi' → 'Uludağ Üniversitesi'`)
- **abbrs**: kısaltma (büyük harf, birebir) → doğru ad (`'ODTÜ' → ...`)
- **foldedAbbrs**: fold edilmiş kısaltma. İki farklı kurumun kısaltması fold edilince çakışıyorsa (ÇÜ = Çukurova, CÜ = Cumhuriyet; ikisi de "cu") bu kısaltma **belirsiz** işaretlenir ve sadece birebir yazıldığında kullanılır.
- **vocab**: sözlükteki tüm kelimeler; kelime kelime yazım düzeltmesi için.

4.3 Bir değeri sözlükte aramak (lookup)

Bir hücre değeri sözlükte sırasıyla şu yollarla aranır; ilk başarılı olan kullanılır:

Sıra	Yöntem	Örnek girdi → çıktı
1	Birebir kısaltma	ODTÜ → Orta Doğu Teknik Üniversitesi
2	Fold edilmiş tam ad	ULUDAG UNIVERSITESI → Uludağ Üniversitesi
3	Kısaltma açma (Üniv., Müh., Fak., Bil.)	Ege Üniv. → Ege Üniversitesi
4	Kelime kelime yazım düzeltme	Uludağ Üniverstesi → Uludağ Üniversitesi
5	Eksik ek ("Üniversitesi" yazılmamış)	Hacettepe → Hacettepe Üniversitesi
6	Ön ek eşleşmesi	Turizm İşletm. → Turizm İşletmeciliği
7	Tüm metne bulanık eşleşme	Benzerlik ≥ 0.85 ve ikinci adaydan en az 0.05 fark

7. adımdaki "0.05 fark" kuralı önemlidir: iki aday birbirine çok yakın puan alıyorsa sistem hangisinin doğru olduğundan emin olamaz ve seçim yapmaz. Bulunan değer benzerlik ≥ 0.9 ise "emin", 0.85–0.9 arasıysa "emin değil" (sarı hücre) işaretlenir.

4.4 Dosyanın kendisinden öğrenme (buildContext)

Sözlükte olmayan bir isim de dosyada doğru yazılmış olabilir. `buildContext` işleme başlamadan önce tüm dosyayı tarar:

- **wordMap**: Türkçe karakter içeren kelimeler (ör. "Gülşah") fold hâlleriyle eşlenir. Aynı dosyada "GULSAH" ya da "Gulsah" görülürse "Gülşah" olarak düzeltilir.
- **Öğrenilen adlar**: Bir ad en az 2 kez geçiyorsa geçerli ad sayılır. **Ama** hazır sözlükteki bir ada 1 harf uzaklıktaysa (ör. dosyada iki kez "Abmet" yazılmışsa) öğrenilmez, çünkü büyük ihtimalle yazım hatasıdır. İstisna: Türkçe harflerle iki kez yazılmışsa (ör. "Miraç") öğrenilir.
- **Türkçe harf koruması**: Bulanık düzeltme Türkçe bir harfi silemez. "Miraç" → "Mira" yapılamaz, çünkü "ç" harfini yazan kişi bilerek yazmıştır.

5. Sütun türünü tanıma (categories.js)

Her sütun 58 kategoriden birine atanır. Kategori; sütundaki her hücreye hangi temizleyicinin uygulanacağını, kişiye ait olup olmadığını (eşleşmede kullanılır mı) ve sütun içi tek yazımın yapılıp yapılmayacağını belirler.

5.1 Kategori tanımı

```
C('phone', 'Telefon', 'İletişim', '"0555 100 00 01" biçimine çevrilir', {
  header: /tel|gsm|cep|phone/,                  // başlıkta bu geçerse
  test:   (v) => /^((\+?90|0)[\s()*[2-5]|...)\d{2}.../.test(v),  // içerik testi
  min:    0.6,                                  // değerlerin %60'ı uyarsa
  why:    'değerler 05… / +90 ile başlayan telefon numarası',
  clean:  cleanPhone,                           // hücre temizleyicisi
  person: true,                                 // kişiye ait: eşleşmede kullanılır
})
```

7 grup vardır: **Kişi** (ad, ad-soyad, TC, doğum tarihi, cinsiyet, kan grubu...), **İletişim** (e-posta, telefon, adres, il, ilçe, posta kodu, web, sosyal medya), **Eğitim** (kurum, fakülte, bölüm, lise, sınıf, not ortalaması...), **İş** (şirket, unvan, departman, maaş, IBAN...), **Kayıt** (kayıt no, kayıt tarihi, durum, kaynak...), **Genel** (tarih, saat, sayı, para, yüzde, evet/hayır, plaka) ve **Diğer**.

5.2 Tanıma sırası (detectColumnType)

- **1. Başlık**: Başlık bir kategorinin düzenli ifadesine uyuyorsa o seçilir. Kelime sınırları kullanılır; "Üniversite" başlığı içindeki "site" yüzünden web adresi sanılmaz (testlerde bulunan bir hataydı).
- **2. İçerik**: Başlık bir şey söylemiyorsa sütundan en fazla 60 dolu değer örneklenir ve kategoriler belirli bir sırayla (`CONTENT_ORDER`) test edilir. Değerlerin yeterli oranı (`min`) uyarsa o kategori seçilir. Örnekler: `@` içeriyor → e-posta; `05xx` → telefon; "Mah./Cad./Sok./No" → adres; "Üniversitesi" → kurum; 11 hane + TC algoritması → TC kimlik; TR + 24 hane → IBAN.
- **Sıra önemlidir**: Şehir, kurumdan önce test edilir (aksi hâlde "Ankara" kurum sanılıyordu); "Dil Seviyesi" dilden önce gelir.
- **3. İsim sezgisi**: Değerlerin çoğu sözlükteki ad/soyadlardan oluşuyorsa ad sütunu kabul edilir.
- **4. Diğer**: Hiçbiri uymazsa genel metin temizliği (boşluk, büyük harf) uygulanır, içerik değiştirilmez.

Her kararın bir **nedeni** saklanır ("başlıkta 'tel' geçiyor" ya da "değerlerin %92'si e-posta biçiminde"). Tablo başlığındaki tür etiketinin üzerine gelince bu neden gösterilir; yanlışsa kullanıcı listeden başka bir tür seçebilir ve seçimi `decisions.types`'a yazılır.

Hız: Tanıma pahalı bir işlemdir ve her karardan sonra tekrar çalışmasına gerek yoktur. Sonuç `WeakMap` önbelleğinde satır dizisine bağlı tutulur; dosya değişmedikçe tekrar hesaplanmaz. WeakMap, dosya kapatılınca kendiliğinden silinir.

6. Hücre temizleyicileri

Her temizleyici bir ham değer alır ve `{ value, sure, known, note }` döndürür: yeni değer, emin olup olmadığı, sözlükte bulunup bulunmadığı ve ne yapıldığına dair kısa açıklama (hücreye tıklayınca görünen not).

Tür	Ne yapar?	Örnek
Ad soyad (cleanName)	Nokta/alt çizgi/camelCase ayırma, "Soyad, Ad" çevirme, Türkçe büyük harf, sözlük ve dosyadan öğrenilenlerle kelime düzeltme	AHMET_YILMAZ → Ahmet Yılmaz; Yılmaz, Ahmet → Ahmet Yılmaz; Abmet Dılmaz → Ahmet Yılmaz
Kurum (cleanInstitution)	Üniversite adını sözlükten bulur; arkasında fakülte varsa onu da ayrıca düzeltir	uludag univ. → Uludağ Üniversitesi
Bölüm / Fakülte	Kısaltma açma (Müh., Bilg., Elk.), sözlük	Bilg. Müh. → Bilgisayar Mühendisliği
Şehir (cleanCity)	Önce takma adlar (ilçe/eski ad → il), sonra 81 il içinde yazım düzeltme	istnbul → İstanbul; Antep → Gaziantep; İzmit → Kocaeli (İzmir değil)
E-posta (cleanEmail)	Küçük harf, boşluk silme, alan adı yazım düzeltmesi	Ali@GMAIL.CON → ali@gmail.com; @gma.il → @gmail.com
Telefon (cleanPhone)	O→0, l/I→1, ülke kodu, 10 haneye indirme, biçim	+90 (555) 1OO 0001 → 0555 100 00 01
Adres (cleanAddress)	Mah., Cad., Sok., No, Kat, Daire kalıpları, ortak kelimeler	ataturk mah. cumh cd no:5 → Atatürk Mah. Cumhuriyet Cad. No: 5
Tarih (cleanDate)	Excel seri sayısı, ISO, g/a/y (., /, -), ay adları	45123 → 18.07.2023; 5 Mart 1998 → 05.03.1998
TC kimlik	11 hane + resmi kontrol hanesi algoritması	Geçersizse emin değil işaretlenir
IBAN	TR + 24 hane, mod-97 doğrulama, 4'erli gruplama	tr12000... → TR12 0001 ...
Plaka, posta kodu, saat, para, yüzde, not ortalaması, yıl, kan grubu, evet/hayır...	Her biri kendi biçimine	34abc123 → 34 ABC 123; a rh+ → A Rh+
Diğer / Not	Sadece boşluk ve büyük harf düzeltilir, içerik korunur	-

6.1 E-posta alan adı düzeltme (fixDomain)

"@" sonrası kısım 19 bilinen alan adıyla (gmail.com, hotmail.com...) karşılaştırılır. Bir aday şu durumlardan biriyle yakalanır: **nokta kaymış/eksik** ("gma.il", "gmailcom", "gmail" → noktalar silinince aynı), **harf hatası** ("gmial.com" → 1 harf; uzun adlarda 2 harf) ya da **uzantı hatası** (".con, .cmo, .vom, .comm"). Bu kurallar her alan adına genel olarak uygulanır. İki önemli kural daha:

- **İlk harf değişikliği hata sayılmaz.** "email.com" bir hata değil başka bir alan adıdır, "gmail.com" yapılmaz.
- **En yakın aday seçilir**; iki aday eşit yakınsa (yandex.con hem yandex.com hem yandex.com.tr'ye yakın) en yakın olan kazanır.
- "mail.com" bilerek listede yok, çünkü gerçek bir servistir ve gmail.com'a 1 harf uzaktadır.

6.2 Gerçek kişi e-postası (humanizeEmail)

Ayarlar'da açıkken e-postalardaki teknik/sistem ekleri genel bir kuralla kaldırılır, sadece "ref" için değil:

```
ece.demirci.ref27@gmail.com   →  ece.demirci@gmail.com     (ref, id, test, tmp, user, no + rakam)
ali+bulten@gmail.com          →  ali@gmail.com             (+etiket)
test.zeynep@gmail.com         →  zeynep@gmail.com          (test/demo/dummy ön eki)
```

Güvenlik kuralı: Sadeleşen adres başka bir kişinin (başka grubun) sadeleşmiş adresiyle aynı olacaksa ikisi de olduğu gibi bırakılır. Böylece iki farklı kişi aynı e-postaya düşmez.

7. Sütun içi tek yazım (harmonizeColumn)

Hücreler tek tek temizlendikten sonra aynı sütundaki değerler birbirine göre de düzenlenir. Sadece "harmonize" işaretli kategorilerde (kurum, bölüm, şehir, unvan, şirket...) yapılır; ad, telefon, e-posta gibi kişiye özel alanlarda yapılmaz.

- **Aynı fold, farklı yazım**: "ankara", "ANKARA", "Ankara" → hepsi en çok kullanılan ve en düzgün yazıma ("Ankara") çekilir.
- **Sözlükte olmayan değerler**: bilinen ya da daha sık geçen bir değere ≥ 0.88 benzerlikteyse ona çekilir. "Ankra Lojistik" (1 kez) → "Ankara Lojistik" (8 kez). Genel ("Diğer") sütunlarda bu sadece sütun kategorik görünüyorsa (az sayıda farklı değer, çok tekrar) yapılır; serbest metinlere dokunulmaz.
- **TAMAMI BÜYÜK harf**: sütundaki çoğu değer normal yazılmışsa büyük harfle yazılmış olanlar da normale çevrilir.

8. Tekrar eden kayıtları bulma (matcher.js)

8.1 Kayıt özeti

Her satırdan karşılaştırma için küçük bir kayıt çıkarılır: `{ name, email, phone, tc, institution, department, address, city }`. Ad ve soyad ayrı sütunlardaysa birleştirilir. Temizlenmiş değerler kullanılır, böylece "AHMET YILMAZ" ile "Ahmet Yılmaz" zaten aynıdır.

8.2 Ad benzerliği (nameSimilarity)

İki adın fold hâlleri karşılaştırılır. Ayrıca kelimeler alfabetik sıralanıp tekrar karşılaştırılır; böylece "Yılmaz Ahmet" ile "Ahmet Yılmaz" de aynı kabul edilir. İkisinden yüksek olanı alınır.

8.3 İki kaydı karşılaştırma kuralları (compareRecords)

Durum	Sonuç
Ad benzerliği < 0.6	Hiç karşılaştırılmaz (farklı kişi)
Telefon aynı, ya da e-posta harfleri ≥ 0.9 benzer ve içindeki rakamlar aynı, ya da TC aynı; çelişki yok; ad ≥ 0.75	**Kesin** (otomatik birleşir)
Kimlik eşleşmesi var ama çok farklı ad (0.6–0.75)	**Şüpheli** (sorulabilir)
Kimlik eşleşmesi var ama başka bir kimlik çelişiyor	**Şüpheli**
Sadece çelişki var (ör. farklı telefon, farklı TC, e-posta < 0.8)	**Farklı**
Kimlik bilgisi yok: ad ≥ 0.9 + aynı kurum + aynı adres	**Kesin**
Kimlik yok: ad ≥ 0.85 + kurum veya adreslerden biri aynı + hiçbiri çelişmiyor	**Şüpheli**
Diğer her şey	Farklı

E-postada rakam kuralı: "ali.kaya1@" ile "ali.kaya2@" harf olarak çok benzer ama içlerindeki rakamlar farklıdır. Bunlar iki farklı Ali Kaya'dır. Bu kural testlerde bulunan yanlış birleştirmeleri sıfıra indirdi.

Ayrıca 0–100 arası bir **puan** hesaplanır (ad × 40, telefon +30, e-posta +30, TC +40, kurum +10, adres +15, bölüm +5, çelişki −40, farklı kurum −15). Puan, şüpheli çiftler arasından kullanıcıya hangisinin sorulacağını ve ekranda gösterilen güven oranını belirler.

8.4 Gruplama (union-find)

`findPairs` tüm satır çiftlerini karşılaştırır. "Kesin" olan çiftler (ve kullanıcının "aynı" dediği çiftler) birer **bağ**tır. `buildGroups` bu bağları **union-find** (birleşim-bul) yapısıyla gruplara çevirir: A=B ve B=C ise A, B, C aynı gruptadır. Her satır başta kendi grubunun lideridir; bir bağ geldiğinde iki grubun liderleri birleştirilir. Bu yöntem çok hızlıdır ve bağların sırasından etkilenmez.

```
parent = [0, 1, 2, 3, 4]          // herkes kendi lideri
bağ (0,2)  → parent[2] = 0
bağ (2,4)  → find(2) = 0, parent[4] = 0
gruplar:  {0,2,4}  {1}  {3}
```

9. Ana işlem hattı (pipeline.js · processTable)

`processTable(headers, rows, decisions)` saf bir fonksiyondur: aynı girdiye her zaman aynı çıktıyı verir ve hiçbir şeyi kaydetmez. Ekrandaki her şey bu fonksiyonun döndürdüğü nesneden çizilir. Kullanıcı her cevap verdiğinde fonksiyon yeni `decisions` ile baştan çalışır; 1000 satırlık bir dosyada bu göz açıp kapayıncaya kadar sürer.

9.1 Adım adım

Adım 1 – Türler

Otomatik bulunan türler alınır; kullanıcı bir sütunun türünü değiştirdiyse (`decisions.types`) o kullanılır. Ardından `buildContext` ile dosyadan öğrenme yapılır.

Adım 2 – Hücre temizliği

Her dolu hücreye kendi türünün temizleyicisi uygulanır. Kullanıcı daha önce bu yazımı elle düzelttiyse (`decisions.rules[sütun][fold(değer)]`) temizleyici çalışmaz, kullanıcının değeri kullanılır. Bu yüzden bir hücreyi düzeltince **aynı yazıma sahip tüm hücreler** birden düzelir.

Adım 3 – Sütun içi tek yazım

Bölüm 7'de anlatılan `harmonizeColumn`.

Adım 4 – Eşleştirme

- Çiftler bulunur. Bağ olarak sayılanlar: kullanıcının "aynı" dediği çiftler + kullanıcının reddetmediği kesin çiftler.
- Kullanıcının "Ayır, farklı kayıtlar" dediği gruptaki satırlar arasında otomatik bağ kurulmaz.
- **Aynı adlı farklı gruplar**: Gruplar oluştuktan sonra aynı ada sahip birden çok grup varsa (iki ayrı "Ahmet Yılmaz") bu sistemin kendi başına karar vereceği bir şey değildir. Kullanıcı "birleştir" dediyse birleştirilir; cevap yoksa Ayıklama ve Manuel modda **"aynı adda N kişi var, birleştireyim mi?"** diye sorulur. Otomatik modda ayrı tutulur.

Adım 5 – Grup içinde en iyi değer

Her grup ve kişiye ait her sütun için gruptaki değerler toplanır ve **kümelere** ayrılır (`clusterVariants`): birbirine ≥ 0.8 benzeyen yazımlar aynı kümededir ("Uludag Üni" ve "Uludağ Üniversitesi"). Telefon, doğum tarihi, TC ve posta kodunda tam eşitlik aranır. Farklı kapı numaralı adresler farklı sayılır. Tek küme varsa sadece yazım farkı vardır; birden çok küme varsa **gerçek çelişki** vardır.

Durum	Ne yapılır?
Kurum, fakülte, bölüm, lise, ilçe, pozisyon (LONGEST_TYPES)	Hiç sorulmaz. En uzun (en ayrıntılı) değer seçilir: "Uludağ Üni" ve "Uludağ Üniversitesi Mühendislik Fakültesi" varsa ikincisi.
E-posta, birden çok küme	Hiç sorulmaz. bestEmail: kişinin adını taşıyan, ref/test/çok rakam içermeyen, bilinen alan adlı adres seçilir.
Diğer çelişkiler, Otomatik/Ayıklama modu	Sorulmaz. pickBest ile çoğunluğun değeri seçilir.
Diğer çelişkiler, Manuel mod	Her satır kendi değerini korur (sarı), "hangisi doğru?" sorusu oluşturulur.
Tek küme (sadece yazım farkı)	pickBest değeri tüm satırlara yazılır; boş hücreler doldurulur (mavi).
Kullanıcı bir seçenek seçti	O değer tüm satırlara yazılır. "Her satır kendi değerini korusun" dediyse dokunulmaz.

pickBest puanı: adayın diğer adaylara benzerlik toplamı (en "ortadaki" yazım kazanır) + sözlükte bulunduysa 1.5 + emin ise 0.5 + Türkçe karakter içeriyorsa 0.5 + e-postada kişinin

adıyla başlıyorsa 1 + kullanıcı değeriyse 100. Böylece "Ahmet Yılmaz", "AHMET YILMAZ" ve "Abmet Dılmaz" arasında hem çoğunluk hem sözlük hem Türkçe harf "Ahmet Yılmaz"ı seçer.

Adım 6 – Son dokunuşlar

- **E-posta nokta onarımı** (`repairEmailDots`): kişinin adı "Can Özdemir" ise "cano.zdemir@" → "can.ozdemir@".
- **Gerçek e-posta biçimi**: Bölüm 6.2 (Ayarlar'da açıksa).
- **Biçimler**: telefonlar seçilen biçime (0555 100 00 01 / 05551000001 / +90 555 100 00 01), tarihler seçilen biçime (05.03.1998 / 1998-03-05 / 05/03/1998).

Adım 7 – Hücre durumları

Her hücre için orijinal ve yeni değer karşılaştırılır: **same** (değişmedi), **fixed** (düzeltildi, yeşil), **filled** (boştu, eşleşen kayıttan dolduruldu, mavi). Emin olunmayan hücreler ayrıca **unsure** (sarı) işaretlenir. Kullanıcının tek bir hücreye yazdığı değer (`decisions.cells`) her şeyin üzerindedir.

Adım 8 – Sorular

Moda göre soru listesi hazırlanır. Sıra: grup onayları → çelişkiler → aynı ad soruları → şüpheli çiftler → hücre soruları. Çelişki soruları öne alınır ki kullanıcı "birleştir" deyince hemen ardından "hangisi doğru?" gelsin. Aynı iki grup için tek soru sorulur, reddedilmiş gruplar tekrar sorulmaz.

9.2 Üç mod

Mod	Ne sorar?	Ne zaman?
1. Otomatik	Hiçbir şey. Şüpheli çiftler ve aynı adlı ayrı gruplar ayrı kalır; çelişkilerde çoğunluk seçilir.	Hızlı sonuç; yanlış birleştirme riski en düşük
2. Ayıklama (varsayılan)	Sadece eşleşme soruları: aynı adlı kişiler, şüpheli çiftler. Ham_Veri'de 2 soru.	Çoğu kullanıcı için ideal
3. Manuel	Her şey: her otomatik grup için onay, her çelişki, emin olunmayan hücreler, tekrarları tek satıra indirme. Ham_Veri'de 34 soru.	Her kararı görmek isteyenler

9.3 Kararlar (decisions) nesnesi

Kullanıcının verdiği her cevap tek bir nesnede tutulur. Algoritma bu nesneyi okuyarak kendini ayarlar:

```
decisions = {
  mode:    'suspicious',            // auto | suspicious | manual
  types:   { 3: 'phone' },          // sütun 3'ün türü elle seçildi
  pairs:   { '4|17': true },        // satır 4 ve 17 aynı kayıt
  names:   { 'ahmet yilmaz': false },  // aynı adlı kişiler ayrı kalsın
  groups:  { '2,9,14': true },      // otomatik grup onaylandı (false = ayır)
  choices: { '2|5': 'Ankara' },     // grup 2'de sütun 5 için doğru değer
  rules:   { 5: { 'ankra': 'Ankara' } },   // bu yazımı her yerde böyle düzelt
  cells:   { '7-3': '0555 ...' },   // tek bir hücreye elle yazılan değer
  review:  'all',                   // "kalanların hepsini onayla"
  dedupe, withGroup, skipped,       // indirme ve soru atlama
  format:  { phone, date, humanEmail }   // Ayarlar'dan gelen biçimler
}
```

Bu tasarımın faydası: **Geri al** çok kolaydır. App.jsx her karardan önce eski decisions'ı bir listeye (en fazla 50) koyar; "Geri al" son elemanı geri yükler ve processTable aynı sonucu yeniden üretir.

10. Arayüz (React bileşenleri)

10.1 App.jsx: durum ve akış

Durum (state)	İçeriği
sheets	Okunan sayfalar: [{ name, headers, rows }] (sadece bellekte)
active	Seçili sayfanın sırası
decisions	Her sayfa için kararlar nesnesi
history	Geri al için eski kararlar (en fazla 50)
settings	Ayarlar (tema, mod, biçimler...). Hiçbir yere kaydedilmez
view	'app' | 'settings' | 'help' (üstteki sekmeler)

`results` bir `useMemo`'dur: sayfalar, kararlar ya da biçim ayarları değişince her sayfa için `processTable` çağrılır. React sadece gerektiğinde yeniden hesaplar.

Önemli fonksiyonlar:

- `handleFile`: dosyayı okur, kararları sıfırlar (varsayılan mod ayarlardan gelir).
- `answer(kind, payload)`: bir sorunun cevabını doğru alana yazar (name → decisions.names, pair → decisions.pairs, choice → decisions.choices...).
- `correct(row, col, original, value)`: hücre balonundan gelen düzeltmeyi kural olarak kaydeder.
- `setType`: sütun türünü değiştirir. `skip` / `unskipAll`: soruyu atla / atlananları geri getir.
- `undo`: geri al. `reset`: her şeyi sil, başa dön.
- `download`: indirme (Bölüm 11).

Tema: `useEffect` ile `<html data-theme>` ve sıkı/kaydırma/renk/ipucu sınıfları ayarlanır. "Sistem" seçiliyse CSS `prefers-color-scheme` ile tarayıcının açık/koyu temasını izler.

10.2 Assistant.jsx: soru balonu

Ekranın üstündeki asistan balonu `result.questions` listesindeki ilk soruyu gösterir. Her soru türünün kendi görünümü vardır:

Soru türü	Ekranda	Butonlar
name	"Aynı adda 3 kişi var: Ahmet Yılmaz" + her kişinin kurum/şehir/telefon özeti	Hepsi aynı kişi / Farklı kişiler
pair	İki kayıt yan yana, benzerlik puanı ve nedenleri	Aynı kayıt / Farklı
group (manuel)	Otomatik bulunan grup, yazım çeşitleri ("Ahmet Yılmaz, Abmet Dılmaz"), önerilen ad	Doğru, birleştir / Ayır
conflict (manuel)	"Hangisi doğru?" + seçenekler ve kaç satırda geçtikleri	Her seçenek / Her satır kendi değerini korusun / kendin yaz
cell (manuel)	Emin olunmayan düzeltme: eski → önerilen	Evet / kendin yaz

Sorular bitince `Done` bileşeni gelir: istatistikler (kaç düzeltme, kaç tekrar), indirme butonları ve atlanan soruları geri getirme. Üstteki mod seçici (`ModeSwitch`) ile mod istendiği an değiştirilebilir; kararlar korunur.

10.3 ResultTable.jsx: temiz tablo

- İlk sütun **Grup**: aynı kayda ait satırlar aynı numarayı alır.

- Her başlığın altında **tür etiketi** vardır. Üzerinde şeffaf bir `<select>` durur; tıklayınca 58 kategori 7 grup hâlinde açılır. Bu hile sayesinde uzun liste tablo genişliğini büyütmez.
- Hücre renkleri CSS sınıflarından gelir: `fixed` yeşil, `filled` mavi, `unsure` sarı. Ayrıca `t-email` gibi tür sınıfları ile e-posta/URL gibi uzun değerlerin satır kırması ayarlanır.
- **Hücre balonu** (CellBubble): bir hücreye tıklayınca açılır; durum, not ("yazım hatası düzeltildi"), dosyadaki hâli, emin değilse "Evet" butonu ve düzeltme kutusu. Kaydedilen değer aynı yazıma sahip tüm hücrelere uygulanır; balon bunun kaç hücre olduğunu söyler.

10.4 Groups.jsx, Settings.jsx, Help.jsx

- **Groups**: "Eşleşen kayıtlar" sekmesi. Her grup bir kart: ad, özet (kurum · şehir · telefon), rozetler ("Aynı adda başka kayıt da var", "Onaylandı", "Otomatik eşleşti"), birleşme nedenleri ("aynı telefon, aynı e-posta"), orijinal satırları gösteren mini tablo ve "Doğru, aynı kayıt" / "Ayır, farklı kayıtlar" butonları.
- **Settings**: 4 grup ayar (Görünüm, Eşleştirme, Biçimler, İndirme). Açma/kapama anahtarları ve seçmeli butonlar. "Varsayılanlara dön". Ayarlar localStorage'a bile yazılmaz; sayfa yenilenince sıfırlanır.
- **Help**: "Nasıl çalışır?" sekmesi; 5 adımda yöntem ve renk açıklaması.
- **Upload**: sürükle-bırak alanı ve gizli `<input type="file">`.

10.5 Açıklama balonları ve tema (styles.css)

Butonların üzerine gelince çıkan siyah balonlar JavaScript değil saf CSS'tir: `data-tip="..."` özelliği olan her öğenin `::after` sözde öğesi bu metni gösterir. Balon sadece fare üzerindeyken çizilir (`display` ile), böylece gizliyken yer kaplamaz ve sayfayı yana kaydırmaz. `[data-tip]:hover { z-index: 100 }` ile soru kutusunun altında kalma sorunu çözüldü. Son sütunlardaki balonlar sağa hizalanır ki ekrandan taşmasın.

Renkler `:root` üzerinde CSS değişkenleri olarak tanımlıdır (`--bg, --text, --accent...`). Koyu temada sadece bu değişkenler değişir. Tablo `word-break` ile sayfaya sığar; sadece e-posta/URL/IBAN gibi tek parça uzun değerlerde her yerden kırılır, normal kelimeler ortadan bölünmez ("Kastamon u" sorunu buydu).

11. İndirme (io.js)

11.1 İndirilecek tabloyu hazırlamak (tableFor)

- **Varsayılan (görev kuralı)**: tüm satırlar olduğu gibi kalır, sadece yazımlar düzeltilir. Değişmeyen hücrelerde orijinal değer (sayı, tarih nesnesi) korunur; böylece Excel'de sayılar metne dönüşmez.
- **Tekrarları tek satıra indir** (`collapseRows`): her grup için ilk satır kalır. Kişiye ait olmayan sütunlarda (kayıt no, kaynak, tarih) gruptaki farklı değerler virgülle birleştirilir: `R001, R012, R059`. Böylece hiçbir bilgi kaybolmaz.
- **Grup no sütunu**: sona `Eslesme_Grubu` sütunu eklenir.

11.2 Excel ve CSV

- **Excel**: `XLSX.utils.aoa_to_sheet` ile her sayfa yeniden oluşturulur, `XLSX.writeFile` tarayıcıda dosyayı üretir. Değişiklik raporu açıksa ek bir "Degisiklikler" sayfası eklenir: sayfa, satır, sütun, eski değer, yeni değer, neden.
- **CSV**: seçilen ayırıcı (virgül ya da noktalı virgül; Türkçe Excel noktalı virgülü sever) ile yazılır. Başına **BOM** (`\uFEFF`) eklenir; bu sayede Excel dosyayı UTF-8 olarak tanır ve Türkçe harfler bozulmaz. Çok sayfalı dosyalarda dosya adına sayfa adı eklenir.
- İndirme `Blob` + geçici bir `URL.createObjectURL` ile yapılır; dosya doğrudan kullanıcının bilgisayarına iner, hiçbir sunucuya uğramaz.

12. Gizlilik: dosya neden hiçbir yerde kalmıyor?

Olası saklama yeri	Durum
Sunucu / API	Yok. Sitede hiçbir fetch/POST isteği yoktur; yüklenen veri dışarı gönderilmez.
localStorage / sessionStorage	Kullanılmıyor (ayarlar dahil).
IndexedDB / Cache / Cookie	Kullanılmıyor.
Service worker	Yok; sayfa çevrimdışı önbelleğe alınmaz.
Analitik / izleme	Yok.
Bellek (RAM)	Veri sadece React durumunda yaşar. Sekme kapanınca ya da "Yeni dosya" denince silinir.

Bu garanti kodun yapısından gelir: `io.js` dosyayı `file.arrayBuffer()` ile okur ve sonucu sadece React durumuna verir; kodda veriyi dışarı gönderen ya da tarayıcıya yazan tek bir satır yoktur. Settings.jsx'in ilk satırındaki yorum da bunu söyler: "Ayarlar sadece bellekte tutulur: tarayıcıya (localStorage vb.) hiçbir şey yazılmaz."

13. Testler

13.1 Motor testi (npm test → tests/evaluate.js)

Doğru cevapları bilinen dosyalar üzerinde motor çalıştırılır ve iki ölçü hesaplanır:

- **Hücre doğruluğu**: doğru değere sahip hücrelerin oranı.
- **Çift metrikleri**: doğru birleştirmeler, kaçırılanlar ve **yanlış birleştirmeler** (en tehlikeli hata: iki farklı kişiyi bir yapmak).

Sorular için bir **sanal kullanıcı** (kahin) vardır: doğru cevabı bildiği için her soruyu doğru yanıtlar (en fazla 200 tur). Böylece her modun kaç soru sorduğu ve sonucun ne kadar iyi olduğu ölçülür.

Test	Sonuç
Ham_Veri.xlsx (görev dosyası), 3 mod	%100 hücre doğruluğu, 0 yanlış birleştirme. Soru sayısı: Otomatik 0, Ayıklama 2, Manuel 34
Sentetik zor senaryolar (ikizler, kimliksiz, başlıksız, ağır kirli)	%97,6 – %99,6 hücre doğruluğu, 0 yanlış birleştirme
Sütun türü tanıma	55 / 55 doğru
Yazım hataları (tests/yazim-hatalari.js)	86 / 86 doğru (80 yazım + 5 gerçek e-posta + 1)
Başlık satırı bulma	Hepsi doğru
Windows-1254 noktalı virgüllü CSV	5 / 5

13.2 Sentetik veri üreticisi (tests/generate.js)

Tohumlu rastgele sayı üreteciyle (her çalıştırmada aynı veri) sahte kişiler üretilir: ad, soyad, kurum, bölüm, şehir, telefon, e-posta. Sonra her kişinin kirli kopyaları yapılır: harf değiştirme, Türkçe harf kaybı, büyük harf, kısaltma, boş hücre, karışık telefon biçimi. Aynı adlı farklı kişiler (ikizler), sözlükte olmayan adlar, ayrı ad/soyad sütunları ve boş başlıklar da eklenir.

13.3 Tarayıcı testi (tests/tarayici-testi.mjs)

Playwright ile gerçek Chromium açılır ve canlı site üzerinde 41 kontrol yapılır: örnek dosya bağlantısının olmaması, test dosyalarını yükleme, üç mod, soruları cevaplama, hücre düzeltme, geri al, sütun türü değiştirme, ayarlar, açık/koyu tema, açıklama balonlarının görünmesi ve soru kutusunun üstünde kalması, tablonun yana taşmaması (1366px masaüstü ve 390px mobil), tekrarları tek satıra indirme, grubu ayırma, Excel/CSV indirme, başlıksız dosyada içerikten tür tanıma, Windows-1254 CSV ve konsolda hata olmaması. Sonuç: **41 / 41**.

13.4 Testlerin yakaladığı hatalar (örnekler)

Hata	Çözüm
Sadece rakamı farklı e-postalar aynı kişi sanıldı	Rakamlar farklıysa farklı kişi
İki kez yazılmış yazım hataları ad olarak öğrenildi	Sözlükteki bir ada 1 harf uzaksa öğrenilmez
ÇÜ / CÜ karıştı	Birebir kısaltma önce; çakışan fold'lar belirsiz
CSV'de 12/11/1975 Amerikan tarihi oldu	raw: true ile okuma
"Üniversite" başlığı web adresi sanıldı	Kelime sınırlı başlık kuralları
İzmit → İzmir, Antakya → Antalya	Şehir takma adları listesi önce
email.com → gmail.com	İlk harf değişikliği hata sayılmaz
Kimlik sütunu yoksa 296 soru	Çelişkide soru yok, grup çifti başına tek soru
Tablo sayfadan taşıyordu	Gizli balonlar yer kaplamıyor, şeffaf tür seçici

14. Yayınlama

- Kod GitHub'da durur (yusufmertd hesabı, kayit-eslestirme deposu).
- Vercel `npm install` ve `npm run build` çalıştırır, çıkan `dist/` klasörünü dünyanın her yerindeki sunucularına dağıtır.
- Yayınlanan şey sadece statik dosyalardır: bir `index.html`, bir JavaScript paketi, bir CSS dosyası ve örnek tablolar. Sunucuda çalışan kod olmadığı için yüklenen dosyayı alabilecek bir yer de yoktur.

15. Baştan sona bir örnek

Şu üç satırlık tablo yüklensin (bu örnek gerçekten motordan geçirildi; aşağıdaki sonuçlar motorun çıktısıdır):

Ad Soyad	Kurum	Telefon	E-posta
Ahmet Yılmaz	Uludag Universitesi	0555 100 00 01	ahmet.yilmaz.ref16@gmail.com
AHMET YILMAZ	Uludağ Üniv. Mühendislik Fak.	+90 555 1000001	
Abmet Dılmaz		05551000001	ahmet.yilmaz@gmial.com

Adım	Ne olur?
Okuma	readFile: 1 başlık + 3 satır. Başlık satırı 0. satır.
Türler	"Ad Soyad" → ad (başlık), "Kurum" → kurum, "Telefon" → telefon, "E-posta" → e-posta.
Temizlik	AHMET YILMAZ → Ahmet Yılmaz; Abmet Dılmaz → Ahmet Yılmaz (iki kelime de sözlükteki ada 1 harf uzak). Uludag Universitesi → Uludağ Üniversitesi; Uludağ Üniv. Mühendislik Fak. → Uludağ Üniversitesi Mühendislik Fakültesi. Telefonlar → 0555 100 00 01. gmial.com → gmail.com.
Eşleşme	1-2: aynı telefon + ad 1.0 → kesin. 1-3 ve 2-3: aynı telefon → kesin. Union-find: tek grup {1,2,3}.
Grup içi seçim	Kurum: hiç sorulmadan en uzun ad seçilir ve 3. satırın boş kurumu doldurulur (mavi). E-posta: iki küme var; bestEmail ad taşıyan ve ref içermeyen ahmet.yilmaz@gmail.com'u seçer, 2. satır doldurulur.
Son dokunuş	Gerçek e-posta açık: ref16 zaten seçilmedi. Telefon biçimi uygulanır.
Sorular	Ayıklama modunda soru yok (eşleşme kesin). Manuel modda "Bu 3 satır aynı kayıt mı?" sorulur.
Ekran	Grup sütununda üç satıra da 1 yazar. Değişen hücreler yeşil, doldurulanlar mavidir.
İndirme	3 satır da kalır, hepsi düzeltilmiş. "Tek satıra indir" açılırsa 1 satıra iner.

16. Ek: dosya dosya önemli fonksiyonlar

Dosya	Fonksiyonlar
text.js	trLower, trUpper, squeeze, isEmpty, titleCase, fold, hasTurkishChars, editDistance, similarity, allowedTypos, closestWord
dictionaries.js	UNIVERSITIES, DEPARTMENTS, FACULTIES, CITIES, CITY_ALIASES, FIRST_NAMES, SURNAMES, TITLES, COUNTRIES, LANGUAGES, STATUSES, EMAIL_DOMAINS, COMMON_WORDS, INSTITUTION/DEPARTMENT/FACULTY/ADDRESS_WORDS, MONTHS, parseList, words
fields.js	buildLookup, buildContext, fixWords, expandedKey, lookup, cleanName, cleanInstitution, isUniversity, isDepartment, cleanDepartment, cleanCity, fixDomain, cleanEmail, humanizeEmail, cleanPhone, cleanAddress, cleanDate, keepAsIs, cleanText
categories.js	CATEGORIES (58), categoryOf, CLEANERS, CATEGORY_GROUPS, CONTENT_ORDER, detectColumnType, isValidTc, cleanIban, cleanPlate, cleanPostal, cleanUrl, cleanSocial, cleanTime, cleanBoolean, cleanMoney, cleanPercent, cleanGpa, cleanYear, cleanBlood, cleanClass, cleanNote
table.js	findHeaderRow
matcher.js	nameSimilarity, emailKey, emailDigits, emailSimilarity, compareRecords, findPairs, buildGroups, pairKey
pipeline.js	processTable, collapseRows, detectAll, clusterVariants, pickBest, emailQuality, bestEmail, longestValue, sameEmail, repairEmailDots, harmonizeColumn, writingQuality, formatPhone, formatDate, mostCommon, summarize, tidyReasons

Dosya	Fonksiyonlar
io.js	decodeCsv, readFile, tableFor, changeReport, downloadXlsx, downloadCsv
App.jsx	App, emptyDecisions, handleFile, decide, undo, answer, correct, setType, skip, unskipAll, setOption, download, trySample, reset, MODES, ModeSwitch, Page, Nav, Steps
components/	Assistant (Question, DedupeQuestion, Done, FreeText, RawRows), ResultTable (CellBubble), Groups, Settings, Help, Upload
tests/	evaluate.js, generate.js, yazim-hatalari.js, tarayici-testi.mjs, make-sample.js, make-blank-header-sample.js

Özet: Site, (1) dosyayı tarayıcı belleğine okur, (2) sütunların türünü başlık ve içerikten anlar, (3) her hücreyi sözlük ve dosyadan öğrenilenlerle temizler, (4) kimlik bilgilerine dayanarak tekrar eden kayıtları bulur, (5) en doğru yazımı seçer, (6) emin olamadığını moda göre sorar, (7) temiz dosyayı yine tarayıcıda üretip indirtir. Hiçbir adımda veri dışarı çıkmaz ya da kaydedilmez.